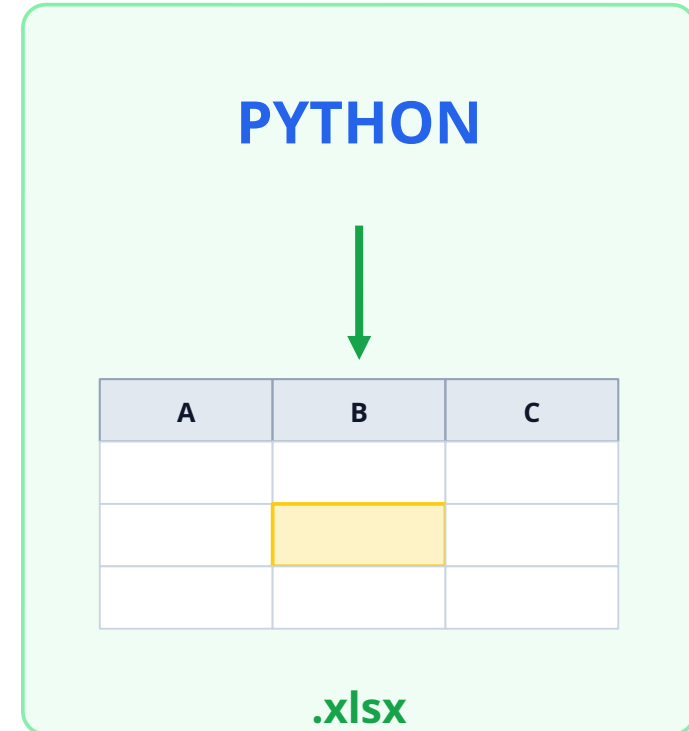


OpenPyXL — Step by Step

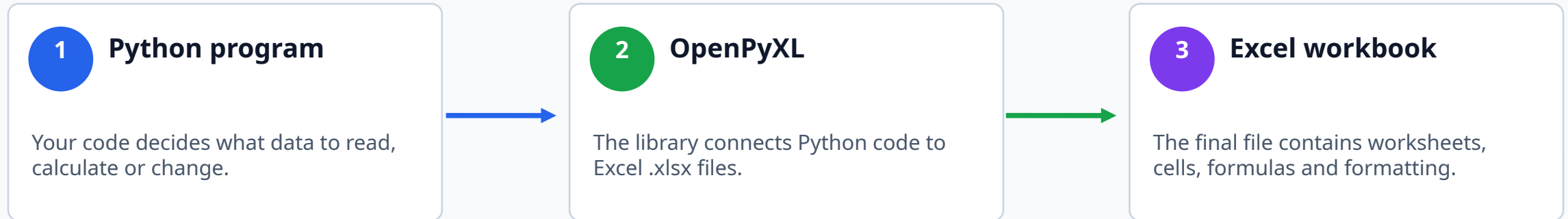
A simple classroom guide to reading, writing and updating Excel files with Python

Goal: by the end, students can build and edit a small .xlsx file themselves.



1. Where does OpenPyXL fit?

Python handles the logic; OpenPyXL handles the Excel workbook.

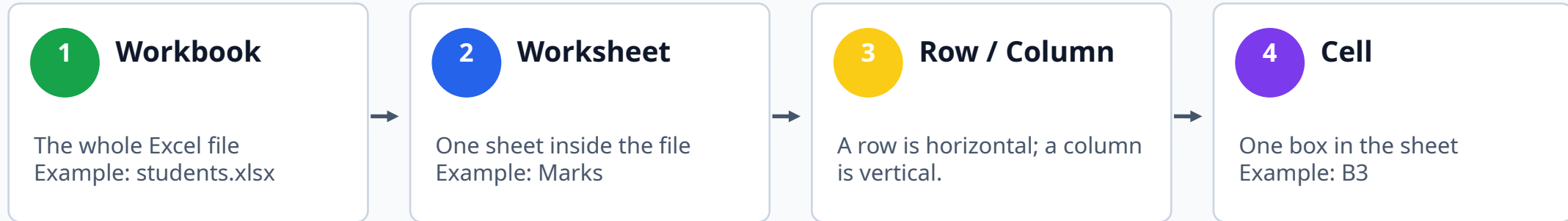


Think of it like this

OpenPyXL is a bridge between Python and Excel.

2. Excel vocabulary first

Before coding, know the four objects you will use most.



A	B	C	D

B3 = column B + row 3

3. Install and import OpenPyXL

Do this once on the computer, then import it in each Python program.

Terminal /
Command
Prompt

```
pip install openpyxl
```

Python

```
import openpyxl  
  
print(openpyxl.__version__)
```

Two different places

pip install runs in the terminal.
import openpyxl runs inside Python code.

Class tip

If import fails, first check that OpenPyXL was installed for the same Python environment you are running.

4. Step 1 — Create a workbook

Start with an empty Excel file in memory.

Python

```
import openpyxl

wb = openpyxl.Workbook()
ws = wb.active
```

What is happening?

wb = the workbook object
ws = the active worksheet object

Workbook → **Active Sheet**

Nothing is saved to disk yet.

5. Step 2 — Name the worksheet and write cells

You can write using cell addresses such as A1, B2 and C5.

Python

```
ws.title = "Marks"

ws["A1"] = "Name"
ws["B1"] = "Python"
ws["C1"] = "Excel"

ws["A2"] = "Aman"
ws["B2"] = 78
ws["C2"] = 84
```

A	B	C
Name	Python	Excel
Aman	78	84

Remember

Text goes in quotes. Numbers do not need quotes.

6. Step 3 — Add many rows with append()

append() is the easiest way to add complete rows.

Python

```
ws.append(["Name", "Python", "Excel"])
ws.append(["Aman", 78, 84])
ws.append(["Simran", 91, 88])
ws.append(["Karan", 67, 72])
```

Why append()?

It adds the list as the next available row.
Very useful when data already exists in Python lists.

Each list → one Excel row

["Simran", 91, 88]



A3 B3 C3

7. Step 4 — Save the workbook

The workbook becomes a real .xlsx file only after `save()`.

Python

```
wb.save("students.xlsx")
```

students.xlsx

saved on disk

Simple rule

Create / change workbook → save it before ending the program.

8. Step 5 — Open an existing workbook

Use `load_workbook()` when the Excel file already exists.

Python

```
import openpyxl

wb = openpyxl.load_workbook("students.xlsx")
ws = wb["Marks"]
```

`load_workbook()`

Reads an existing .xlsx file into Python.

`wb["Marks"]`

Selects the worksheet named Marks.

Common error

If the filename or sheet name is wrong, Python will raise an error.

9. Step 6 — Read cell values

Use `.value` to get the actual content of a cell.

Python

```
name = ws["A2"].value
python_marks = ws["B2"].value

print(name)
print(python_marks)
```

A2 → "Aman"

B2 → 78

Also possible

`ws.cell(row=2, column=2).value` gives B2.

10. Step 7 — Read many rows using a loop

A loop saves you from writing A2, A3, A4... manually.

Python

```
for row in range(2, ws.max_row + 1):  
    name = ws.cell(row=row, column=1).value  
    marks = ws.cell(row=row, column=2).value  
    print(name, marks)
```

Loop values

```
row = 2  
row = 3  
row = 4  
row = 5
```

Why start at 2?

Row 1 usually contains headings.

11. max_row and max_column

OpenPyXL can tell you the used size of the worksheet.

Python

```
print(ws.max_row)
print(ws.max_column)
```

Useful for loops

You do not need to know in advance how many rows are in the file.

A	B	C	D

max_row = 5

max_column = 4

12. Step 8 — Update existing cells

Read the workbook, change a value, then save again.

Python

```
wb = openpyxl.load_workbook("students.xlsx")
ws = wb["Marks"]

ws["B2"] = 85

wb.save("students.xlsx")
```

Before

B2 = 78



After

B2 = 85

Important

Changing the Python object is not enough —
save() writes the change back to the file.

13. Step 9 — Write Excel formulas

A formula is written as a string that begins with =

Python

```
ws["D1"] = "Average"  
ws["D2"] = "=AVERAGE(B2:C2)"  
  
wb.save("students.xlsx")
```

OpenPyXL writes the formula

Excel calculates the result when the workbook is opened/recalculated.

D2 contains

=AVERAGE (B2 : C2)

14. data_only=True

Use it when you want the saved result of a formula instead of the formula text.

Normal load

Python

```
wb =  
openpyxl.load_workbook("students.xlsx")  
print(wb["Marks"]["D2"].value)
```

Output: =AVERAGE(B2:C2)

With data_only=True

Python

```
wb = openpyxl.load_workbook(  
    "students.xlsx",  
    data_only=True  
)  
print(wb["Marks"]["D2"].value)
```

Output: 81.0 (example)

Important

OpenPyXL does not calculate formulas itself. It reads the value saved by Excel the last time the file was calculated.

15. Simple formatting

Formatting is optional. Learn the data first, then make it look good.

Python

```
from openpyxl.styles import Font, PatternFill

ws["A1"].font = Font(bold=True)
ws["A1"].fill = PatternFill(
    "solid", fgColor="D9EAF7"
)

ws.column_dimensions["A"].width = 20
```

Name
Aman

Keep it simple

For beginners, focus on Font, PatternFill and column width.

16. Mini project — Student marks workbook

Let's combine the basics into one small useful program.

Python

```
import openpyxl

wb = openpyxl.Workbook()
ws = wb.active
ws.title = "Marks"

ws.append(["Name", "Python", "Excel"])
ws.append(["Aman", 78, 84])
ws.append(["Simran", 91, 88])
ws.append(["Karan", 67, 72])

wb.save("students.xlsx")
```

1 Create

Workbook + active sheet

2 Add data

Use `append()` for each row

3 Save

Create `students.xlsx`

This is already a complete OpenPyXL program.

17. Average a column — step by step

This is the same pattern as your class example.

Python

```
def average_column(filename, sheet_name, column):  
    wb = openpyxl.load_workbook(filename, data_only=True)  
    ws = wb[sheet_name]  
    values = []  
  
    for row in range(2, ws.max_row + 1):  
        value = ws.cell(row=row, column=column).value  
        if value is not None:  
            values.append(value)  
  
    return sum(values) / len(values)
```

1. Open workbook

2. Select sheet

3. Collect values

4. Calculate average

Skip row 1 because it usually contains headings.

18. Optional next step — Create a chart

Once students are comfortable with cells and ranges, charts become easier.

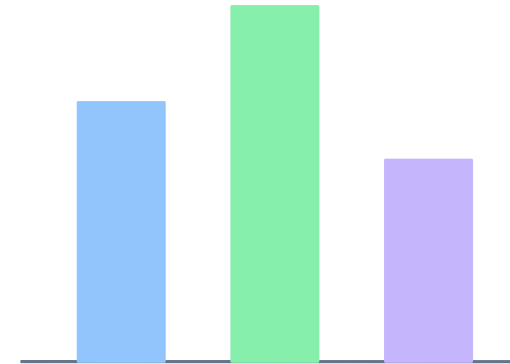
Python

```
from openpyxl.chart import BarChart, Reference

chart = BarChart()
data = Reference(ws, min_col=2, min_row=1, max_row=4)
chart.add_data(data, titles_from_data=True)
ws.add_chart(chart, "F2")

wb.save("students.xlsx")
```

Example chart



Do this later

Charts are useful, but students should first be comfortable with workbooks, worksheets and cells.

19. Common errors and quick fixes

Most beginner problems are small naming or file-path mistakes.

1 **FileNotFoundError**

Check filename, extension and folder.

2 **KeyError for sheet**

Check the exact worksheet name.

3 **Forgot to save**

Call `wb.save(...)` after changing data.

4 **None values**

The cell may be empty or a formula result may not be saved yet.

5 **Wrong row/column**

Remember: row first, column second in `ws.cell()`.

6 **Text mixed with numbers**

Validate values before using `sum()` or averages.

Debugging habit

Print the filename, sheet name and the values you collected. Small prints often reveal the problem immediately.

20. Practice task

Build this in class without copying the complete answer.

Create a workbook named `sales.xlsx` with a worksheet called `Sales`.

1 Headings

Product, Quantity, Price

2 Add 3 rows

Use `append()` to enter sample sales.

3 Read values

Print every product and quantity.

4 Change one value

Update one price and save again.

5 Bonus

Add a Total formula = Quantity × Price.

21. OpenPyXL cheat sheet

The six lines students should remember first.

Python

```
import openpyxl

wb = openpyxl.Workbook()
ws = wb.active
ws["A1"] = "Hello"
wb.save("demo.xlsx")

wb = openpyxl.load_workbook("demo.xlsx")
ws = wb.active
print(ws["A1"].value)
```

- Workbook = Excel file
- Worksheet = one sheet
- Cell = one box such as A1
- Use .value when reading
- Use append() for rows
- Always save after changes

Official reference

openpyxl.readthedocs.io